

The Quant Developer Roadmap

From your existing quant/ML portfolio to job-ready — a 12-week intensive plan

Built for Shantanu — extending your existing algorithmic-trading system and EquityLens work into a quant developer skill set, benchmarked against real quant developer postings currently live in the London market (Millennium, JPMorgan, Legal & General, and specialist recruiter overviews).

How to Use This Document

This isn't a syllabus to read — it's a set of checkpoints to pass. Each stage below gives you:

- **Skill targets**, organised by area (quant/math, engineering, financial instruments, production practice), scoped to what actually gets tested in quant developer interviews and probation periods.
- **Principles**, the reasoning behind the skill — the thing that separates someone who called a library function from someone who understands why the model or the code is shaped that way.
- **A checkpoint**, a concrete, buildable increment on your existing trading system (your “spine project” — see below). If you can't do the checkpoint, you're not ready to move on, regardless of how the calendar looks.
- **Self-check questions**, the kind a quant interviewer would actually ask — about statistics, about your code, and about your own modelling choices. Answer them out loud, unaided, before you consider a stage done. If you have to look it up, that's your revision list.

A note on pace: 12 weeks intensive is realistic for you specifically because you're not starting from zero on the quant side — you already have a live-capital algorithmic trading system (XGBoost/LightGBM, GARCH, Fama-French ensemble), EquityLens (LSTM, walk-forward backtesting, MLflow), a finance-module NLP trading project, professional C#/.NET engineering experience, and Azure DevOps CI/CD exposure. That's genuinely more than most junior quant candidates walk in with. This plan assumes roughly 15–20+ focused hours a week. If a week slips, extend the timeline rather than compressing later stages — in quant interviews, sloppy fundamentals get found immediately.

A note on AI tools: use them to scaffold boilerplate, explore unfamiliar libraries, and speed up literature-style research — but never let one hand you a numerical result (a Greek, a risk metric, a backtest number) you haven't independently verified. In quant work, an AI tool that's subtly wrong about a number is far more dangerous than one that's subtly wrong about a React component. Verify against a known-correct reference or a second method before you trust it.

Where You Are Now

You self-assessed as “comfortable with the basics, a few personal projects shipped.” Against a general software-engineering bar that's a fair, even slightly conservative, read — but against a quant developer bar specifically, it understates you. Real quant postings ask for exactly the kind of work you've already done:

- **Genuine strengths:** a live-capital-bound algorithmic trading system with an ensemble architecture (XGBoost/LightGBM, GARCH, Fama-French) and Interactive Brokers as the execution venue; EquityLens, an LSTM stock-prediction project with proper walk-forward backtesting and MLflow tracking; an ECOM217 group project building an NLP sentiment trading strategy evaluated against a Fama-French 3-factor alpha — including correctly diagnosing a near-zero-return result as an NLP recall failure rather than dressing up the numbers, which is exactly the intellectual honesty quant

interviewers screen for; professional C#/.NET engineering with real Azure DevOps CI/CD ownership; and an MEng in Electronic Engineering, a genuinely quantitative degree.

- **Likely gaps (typical for this profile):** C++ — it shows up alongside Python in most quant developer postings you'd be targeting, and you haven't used it yet. Formal derivatives/fixed-income pricing theory (Black-Scholes, Greeks, yield curves) beyond the equities/factor-model work you've done. Classic CS/data-structures-and-algorithms interview fluency, which quant developer screens test directly. And production-grade practices specific to quant code — numerical testing, reproducibility, CI/CD — pointed at the trading system itself rather than a work codebase.

Baseline self-check — do this before Stage 1

If you can answer all of these confidently and correctly, skim Stage 1 and move faster. If two or more give you trouble, that's exactly where Stage 1 should slow down for you.

- ? Explain what a (weakly) stationary time series is, and why your GARCH model needs one — how would you actually check that in your own data pipeline?
- ? Explain, precisely, why your Fama-French factors were a reasonable choice over CAPM alone — not just that they're standard.
- ? What's the Big-O of a naive nested-loop rolling calculation over your price history, versus the vectorised numpy equivalent — and can you point to a place in your own code where this matters?
- ? If future data leaked into one of your model's features by accident, where in your own pipeline would you look first, and how would you catch it before it reached a backtest result?

The 12 Weeks at a Glance

Stage	Focus	Checkpoint
1	Quant & CS foundations, verified — stats/time series, linear algebra, DSA, vectorisation, first contact with C++	Vectorised rewrite of a rolling calc + DSA screening bar + a written justification of your model choices
2	Financial engineering — derivatives, fixed income, portfolio/risk metrics, backtesting rigour	Risk-metrics module + a hand-verified option price + a lookahead-bias audit
3	Systems & production quant engineering — C++ hot path, market data/infra, numerical testing, CI	One hot-path routine ported to C++ with Python bindings, benchmarked, tested, and in CI
4	Portfolio, domain fluency, interview readiness	Polished write-ups, mock interview, applications go out

Your Spine Project

Unlike a general software-engineering roadmap, here you don't start a new project — you deepen the one you already have. For quant developer roles, your existing algorithmic trading system and EquityLens are the differentiator; the goal of these 12 weeks is to make them rigorous, fast, and defensible under technical questioning, not to build something unrelated from scratch.

Treat the live trading system as the primary spine (it already touches execution, an ensemble model, and a live dashboard — the richest surface for the checkpoints below), and use EquityLens and the ECOM217

project as secondary evidence in your portfolio write-ups. Each stage below adds a specific, real increment to the trading system rather than a toy exercise bolted on beside it.

Stage 1 — Weeks 1–3 Quant & CS Foundations, Verified

The goal here is to firm up the mathematical and computer-science bedrock underneath the trading system you've already built. A lot of quant developer interviews test exactly this layer directly, independent of how good your finance-specific work is — so it has to be solid on its own.

Skill targets

- **Stats & time series:** Stationarity, ACF/PACF, hypothesis testing, and why GARCH models volatility clustering specifically — tie this directly back to why you chose it. Walk-forward validation versus standard k-fold, and why shuffling a time series for cross-validation leaks information from the future.
- **Linear algebra:** Matrix operations, eigen-decomposition and PCA intuition, and covariance matrices — the machinery underneath factor models like Fama-French, whether or not you've thought of it that way before.
- **CS fundamentals:** Arrays, hashmaps, trees, and Big-O well enough to comfortably clear a 45-minute screening round — quant developer interviews test this directly, not just quant researcher ones.
- **Programming depth:** Numpy/pandas vectorisation discipline (eliminate Python loops from hot paths), and first real contact with C++ — syntax, compilation, STL containers, and enough of the memory model (pointers/references) to read code, not yet to ship it.

Principles to internalise

- Stationarity is a modelling assumption, not a formality — check it in your own data, don't assume it.
- Walk-forward validation exists because standard k-fold shuffling breaks the arrow of time — a model that looks good under k-fold on price data may be seeing the future.
- Vectorise before you reach for a compiled language — most “slow Python” in quant code is an unvectorised loop, not a language problem.
- Big-O still matters in research code — an $O(n^2)$ backtest becomes a real bottleneck the moment your price history grows.

Checkpoint — Stage 1

Take one loop-heavy piece of your existing trading system (a rolling-window calculation is a good candidate) and rewrite it fully vectorised in numpy, benchmarked against the original with real timing numbers. Separately, solve enough classic data-structures/algorithms problems (arrays, hashmaps, trees — easy/medium tier) to comfortably pass a 45-minute screening round. Write a short note justifying, in statistical terms, why GARCH and Fama-French were reasonable choices for your system — not just that they're standard.

Self-check questions

- ? Why does your GARCH model need a (weakly) stationary series, and how do you check that in your own data pipeline?

- ? Walk through why k -fold cross-validation is inappropriate for your price-prediction model, and what walk-forward validation does instead.
- ? What's the Big-O of your original rolling-window loop, and what is it after vectorising?
- ? Explain eigen-decomposition well enough to say what a factor model like Fama-French is actually doing to the covariance structure of returns.

Stage 2 — Weeks 4–6 Financial Engineering & Instruments

Your factor-model and ML work already covers equities well. This stage extends your instrument knowledge into where a lot of quant developer roles actually live — derivatives, fixed income, and rigorous risk measurement — and tightens your backtesting discipline.

Skill targets

- **Derivatives fundamentals:** Black-Scholes assumptions and the logic behind the derivation (not necessarily a full stochastic-calculus proof), what each of the Greeks means operationally, and put-call parity as a sanity check you can run on any option-pricing code you write.
- **Fixed income basics:** Yield curves, duration and convexity, and a working explanation of why bond prices move the way they do when rates shift.
- **Portfolio & risk:** Markowitz mean-variance intuition, extending your existing Fama-French familiarity toward a broader factor-model view, and implementing proper risk metrics — VaR, Sharpe/Sortino, max drawdown — if your trading dashboard doesn't already report them.
- **Backtesting rigour:** Formalise lookahead-bias checks, transaction-cost and slippage modelling, and out-of-sample reporting discipline — the same instinct you showed diagnosing the ECOM217 sentiment model's recall failure rather than dressing up the return numbers. That's exactly the muscle to keep exercising here.

Principles to internalise

- A pricing model is a set of assumptions wearing an equation — know which ones you're relying on, and where they break in real markets.
- Risk-adjusted return, not raw return, is the currency quant interviews and quant desks actually think in.
- A backtest without transaction costs and slippage is a fiction — the real edge is whatever survives those.
- Diagnosing a bad result honestly is worth more to an interviewer than a good result you can't explain.

Checkpoint — Stage 2

Add a proper risk-metrics module to your trading system if it doesn't already have one — VaR, Sharpe/Sortino, max drawdown, reported alongside every backtest run. Price one vanilla option in code using Black-Scholes and verify it against a known-correct reference implementation. Write a short lookahead-bias audit of your existing backtest code — find and fix at least one place data could leak from the future, even if it's a near-miss.

Self-check questions

- ? What are the assumptions behind Black-Scholes, and which one is most obviously wrong in real markets?
- ? Explain duration and convexity well enough to say what happens to a bond portfolio when rates move 100bps.
- ? Where exactly could lookahead bias creep into your own feature engineering pipeline, and how would you catch it in code review?
- ? Why is Sharpe ratio alone not enough to judge your strategy, and what would you look at alongside it?

Stage 3 — Weeks 7–9 Systems, Performance & Production Quant Engineering

This is the stage that turns “I can build a model” into “I can build something a desk would actually run.” Python plus C++ is the most common pairing in real quant developer postings — Millennium, JPMorgan, and most sell-side/buy-side listings ask for both. You don't need C++ mastery, but you do need real contact with it.

Skill targets

- **C++ for quant:** Implement one performance-critical routine from your trading system in C++ (a rolling calculation or the core of your ensemble scoring loop is a good candidate), and get comfortable with basic Python/C++ interop such as pybind11 — the pattern most shops actually use, not a full rewrite of everything.
- **Market data & infra:** Understand tick versus bar data, get basic exposure to a time-series-oriented datastore, and learn enough market microstructure — order types, bid-ask spread, sources of slippage — to reason about why your execution assumptions matter.
- **Production practices:** Numerical and property-based testing — not just unit tests, but tests that your Black-Scholes implementation holds put-call parity, or that your risk metrics behave correctly on synthetic data — plus reproducibility (seeded randomness, pinned environments) and a CI pipeline for the trading system itself. Lean directly on your Azure DevOps background here; the concepts transfer, just pointed at this codebase.
- **Performance awareness:** What the GIL actually means for your Python trading code, when it genuinely matters and when it doesn't, and whether multiprocessing or a C++ extension is the right fix for a given bottleneck.

Principles to internalise

- Floating-point arithmetic isn't associative — summation order can change results at the margin, and margins are exactly what finance cares about.
- Determinism is non-negotiable in production trading code — the same inputs must produce the same outputs every time, or you can't debug it.
- Reach for C++ (or Numba/Cython) only where profiling shows you need it — premature rewriting is wasted effort in the wrong place.
- Your execution assumptions — fills, slippage, latency — are part of the model, not an afterthought bolted on at the end.

Checkpoint — Stage 3

Reimplement one hot-path computation from your trading system in C++, wire it back into your Python pipeline via a binding layer, and benchmark it properly against the pure-Python version. Add property-based tests for at least one pricing/risk function (e.g. put-call parity should hold within tolerance). Stand up a CI pipeline for the trading-system repo — lint, test, and a backtest smoke-test on every push.

Self-check questions

- ? Walk through what your Python/C++ binding does at the interface — what gets marshalled across, and what's the performance cost of that boundary itself?
- ? Where in your system would a floating-point rounding difference actually matter, and where would it not?
- ? What does “reproducible” mean for a backtest, concretely — what has to be pinned or seeded for two runs to agree exactly?
- ? What does your CI pipeline actually catch that manual testing wouldn't?

Stage 4 — Weeks 10–12 Portfolio, Domain Fluency & Interview Readiness

Technically you're close to done by now. This stage is about making that legible to someone who has an hour to interview you — and about being ready to defend, not just describe, every choice in your own system.

Skill targets

- **Business communication:** Translate your system into what a hiring manager actually wants to hear — capacity, risk-adjusted return, drawdown behaviour — rather than “it uses XGBoost.” Practise presenting the ECOM217 result (a strategy that didn't work, correctly diagnosed) as a strength, not something to hide.
- **System design (quant flavour):** Be able to whiteboard, at a high level, how a market-data-ingestion → signal → execution pipeline fits together, and where your own system sits on that map.
- **Interview mechanics:** Quant-specific prep alongside general DS&A — probability brainteasers, mental maths under pressure — and being ready to defend every modelling choice in your own projects: why GARCH and not a simpler volatility measure, why Fama-French and not CAPM alone.
- **Portfolio polish:** Case-study write-ups for EquityLens and the trading system that lead with risk-adjusted metrics and honest limitations, not just headline returns.

Principles to internalise

- An interviewer trusts a candidate who states a strategy's weaknesses unprompted far more than one who only presents the upside.
- “Why this model and not a simpler one” is asked in almost every quant interview — have a real answer, not a rehearsed one.
- Business framing — capacity, drawdown, risk-adjusted return — is what turns “a cool ML project” into “something a desk would fund.”

Checkpoint — Stage 4

Run one mock interview mixing DS&A/CS questions with markets/quant questions and a deep dive into your own project internals. Finish polished write-ups of EquityLens and the trading system, each leading with a risk/performance summary. Applications go out this week, not “when it feels perfect.”

Self-check questions

- ? Pitch your trading system's edge and risk profile in 90 seconds to a portfolio manager, then defend the model choice under real pushback.
- ? What's the weakest assumption in your system, and what would you do about it with another month?
- ? Why GARCH and not a simpler rolling-volatility measure — what's the actual justification?

Appendix A: What Real Quant Developer Postings Ask For

This roadmap is checked against current London quant developer postings (Millennium, JPMorgan's Quantitative Trading & Research desk, Legal & General's junior quant developer role, and specialist recruiter overviews) rather than a single job description, since no one posting captures the range you'd realistically be applying to:

Requirement area	What London postings ask for	Where you stand
Languages	Python is near-universal; C++ appears in most postings alongside it, especially front-office/market-making. C#, Java and Rust show up as accepted alternates at some shops.	You have strong Python and professional C#/ .NET — a real asset most junior candidates don't have. C++ is your genuine gap; Stage 1 and 3 address it directly.
CS fundamentals	Data structures, algorithms, Big-O, OOP, and system-design basics — tested directly in screens, not assumed from a CS degree.	Covered in Stage 1; your MEng gives you the maths base, this fills the interview-specific gap.
Financial products	Equities, FX, fixed income, credit, derivatives/volatility — postings usually want fluency in at least one or two asset classes.	You're strong on equities/factor models already; Stage 2 extends you into derivatives and fixed income.
Numerical & tooling	numpy/pandas, SQL/NoSQL, git, Linux, CI/CD, cloud — explicitly listed even in “research-flavoured” postings.	Largely covered by your existing Python work and Azure DevOps background; Stage 3 points it specifically at the trading system.
Statistics & ML	Regression, time series, and increasingly machine learning for signal generation — a genuine growth area across postings.	This is your strongest area already, via EquityLens and the ensemble trading system.

The pattern across almost every posting is the same: strong programming (Python plus something compiled), real fluency in at least one or two financial products, and comfort with the maths underneath your own models — not just calling a library function. That's the shape this roadmap is built around.

Appendix B: After Week 12

Being “job-ready” at the end of this plan means: interview-ready for junior/junior-mid quant developer roles, with a live-capital trading system and EquityLens you can defend in depth, and the vocabulary to discuss model and engineering trade-offs like a quant, not a tutorial-follower. It doesn't mean finished learning — nobody is, least of all in quant finance. Two habits worth carrying forward past week 12:

- **Keep the trading system alive.** Small, deliberate additions — a new risk metric, a new factor, a faster hot path — teach you more over months than a new toy project every few weeks.
- **Read papers and real code weekly.** Pick one quant-finance paper or one well-regarded open-source quant/ML repo and go through it properly each week — it's the fastest way to absorb how practitioners actually reason, beyond what a solo project teaches you.